Canonical Profile Transformer: Technical Design

Mithil Batra · mithil.batra05@gmail.com

The same candidate turns up in a recruiter CSV, an ATS export whose field names look nothing like ours, a GitHub profile, and free-text recruiter notes. The sources are partial, sometimes malformed, and they disagree. I fold them into one profile per person with fixed fields and normalized formats, recording where every value came from and how much to trust it. The design turns on one idea: a confidently wrong value is worse than an empty one, since it quietly corrupts hiring decisions downstream. When the engine cannot be sure, it writes `null` rather than guess.

Pipeline

Extractors only lift raw values out of a format; they never normalize, merge, or score. Every source produces the same observation `{field, value, method}`, grouped into one subject per person, so a CSV row and a scraped GitHub page sit on identical footing and the rest of the pipeline keeps a single code path.

`each source emits a uniform observation {field, value, method}; the canonical record is built once, then projected per config`

Canonical schema

```
candidate_id, full_name, emails[], phones[],
location{city,region,country},
links{linkedin,github,portfolio,other[]},
headline, years_experience,
skills[{name,confidence,sources[]}],
experience[{company,title,start,end,summary}],
education[{institution,degree,field,end_year}],
provenance[{field,source,method}],
overall_confidence
```

Normalized formats

`phones`	E.164 via libphonenumber, US assumed when no country code
`dates`	YYYY-MM; 'Present' becomes null; bare year pinned to -01
`country`	ISO-3166 alpha-2 (USA to US, U.K. to GB, 'City, TX' infers US)
`skills`	alias table to one canonical name; unknown kept and flagged
`links`	split into linkedin / github / portfolio / other

Merge: who is the same person

Union-find over the subjects decides who is one person, joining only on keys I trust: a normalized email, an E.164 phone, or a profile URL. The GitHub record has no email or phone, so alone it would float free, but its profile link matches the same link in the ATS blob and that one key pulls it into the right cluster. Name plus current company is a weaker join; a name by itself never joins, since at thousands of candidates it produces too many false merges.

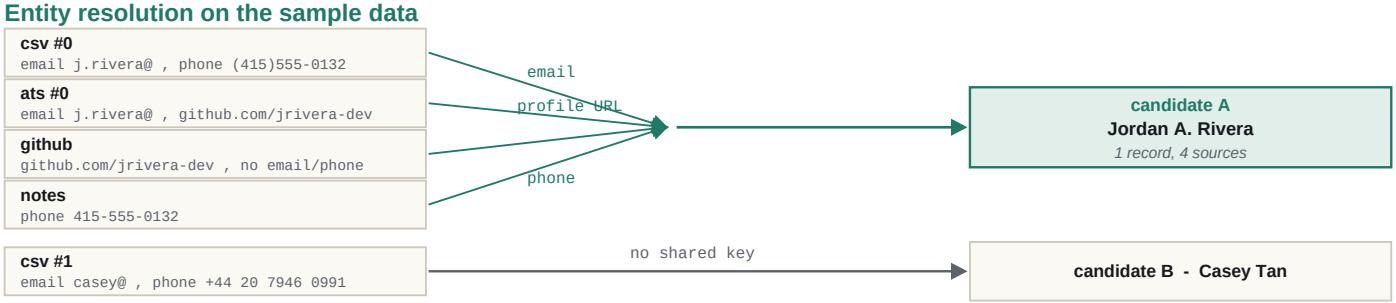
Merge: which value wins

Within a cluster each observation has a weight of source reliability times field affinity. Highest weight takes the field; ties fall back to agreement count, then a stable id, so output never depends on the order files were read. Scalars pick one winner; lists like emails and skills keep the deduplicated union.

Confidence

Affinity encodes that a source is reliable for some fields and not others: GitHub is strong on languages and links, weak on phone numbers; the ATS owns contact details. Agreement then lifts the score, each repeat closing part of the gap to 1.0, so a skill in three sources lands near 0.93 while a notes-only mention stays near 0.28. Overall score is `0.6 * mean(field) + 0.4 * core completeness`.

Entity resolution on the sample data



Four subjects collapse into Jordan because each shares a strong key with another. Casey is in the same CSV but shares no key with that cluster, so she stays a separate record instead of a wrong guess.

Runtime output config: projection and validation

The canonical record is immutable. A projection layer reads it through a config and assembles a fresh object, so a new output shape never touches engine code. A config selects fields, remaps them from a canonical path (`emails[0]`, `location.country`, the `skills[].name` fan-out), sets a per-field normalize (E164, canonical, case), toggles provenance and confidence, and decides what a missing value means: write null, omit the key, or raise. Whatever it builds is validated against the declared field set and types before it returns.

Edge cases

Case	Handling
Malformed or empty source	Caught per source and reported with a reason; the run keeps going and still resolves every other profile.
Same person, conflicting values	Folded into one record; winner by weighted source-times-field, ties broken deterministically, every value kept in provenance.
GitHub record with no email or phone	Joins via the shared profile URL; if nothing links it, it stays its own profile instead of a bad merge.
Two different people in one file	Stay distinct; differing contact keys block the merge and name similarity alone is never enough to join.
Unknown skill or unparseable date	Skill is title-cased, kept, flagged known:false so its confidence is dampened; a bad date becomes null, not an invented month.

What I left out, on purpose

Resume PDF and DOCX parsing: the observation IR makes that one more extractor, not an engine change, and CSV, ATS, GitHub, LinkedIn and notes already exercise every path. Also descoped: fuzzy name matching beyond the normalized keys, and persistence, auth and streaming for large batches. The time went to what the brief weighs most: the merge and confidence logic, and explainable, reproducible output.